$ALPS$: Admissible Locality and Payload Semantics for Region-Based Parallel Mining

M Mohaiminul Islam
Department of Computer Science & Telecommunication Engineering

***Abstract*—In independent-barrier execution, regions validate against one starting state and merge accepted changes additively. Payload clustering can then split debits of one balance across regions. We prove that guarded-key locality is necessary and sufficient for balance non-negativity in this model. ALPS enforces the stronger sender locality required by account-wide Ethereum nonces and optimises semantic locality, active-account interaction cut, and historical load.**

Across four direct-call traces from Ethereum and BNB Smart Chain (145,372 submitted calls), ALPS improves retrieval locality and interaction cut over address hashing in all 24 trace/region settings. Compared with balanced sender k-means, it lowers the interaction cut in all 24 settings by 12.8–82.5%; R_{90} improves in 19 settings, ties in two, and is worse in three. In a prospective 70/30 holdout, ALPS scans 4,404 rows at 90.78% suffix recall, compared with 8,312 rows at 88.46% for METIS+C, but has a higher interaction cut (.592 versus .097) and load (1.534 versus 1.191). Interpolation of frozen plans at 90% recall estimates 4,419 rows for ALPS and 7,139–9,020 for the comparators. These scan-work results support ALPS for retrieval-oriented workloads whose savings justify its interaction, load, and allocation costs; they do not establish end-to-end latency gains.

***Index Terms*—blockchain, parallel mining, transaction partitioning, clustering, state conflicts, Ethereum**

I. Introduction

State-based blockchains often process transactions in sequence because validity depends on shared state [1]. Region-based parallel mining [2] divides pending transactions among independent regions. Each region processes its assigned transactions, and the regional blocks are joined at a barrier. This approach can increase parallel work, but it also creates a placement problem. During each round, regions validate transactions against the same committed state. Two regions may therefore accept debits that become unsafe when their changes are merged.

Semantic sharding improves retrieval [3], while workload-aware placement reduces cross-shard traffic [4]–[6] and conflict-oriented systems change ordering or scheduling [7], [8]. These lines do not characterise admissible placement for independent barriers.

Grouping calls by payload can keep related token operations close and reduce the number of regions scanned by an asset query. However, payload describes the operation, not the balance that it uses. Two regions may validate debits of the same account-token balance against the same starting state. The merged balance can then become negative. We call the required same-region rule *guarded-key locality*. Ethereum's transaction nonce is account-wide, not token-specific. Keeping each guarded key local therefore does not always keep the sender's nonce order local. ALPS uses the stronger rule of assigning all calls from one sender to one region.

The main contributions of this paper are:

- We characterise balance-safe partitions for concurrent, same-snapshot regional execution with additive merging and no cross-region serialization: all debits of each guarded key must be region-local.
- We present ALPS, a sender-indivisible allocator that combines weighted semantic prototypes, an interaction-graph cut, and load control under a hard static historical-load constraint. Exact whole-sender moves give monotone local refinement; a chronological extension pins new senders.
- We evaluate safety, scheduling, retrieval, interaction, load, and allocation cost on four direct-call traces (145,372 calls) and a separate 17,445-event log trace, including prospective replay, ablations, METIS, and a reimplementation of the TxAllo allocation objective.

The novelty lies in deriving the admissible design space, separating guarded-key safety from account-nonce locality, and making that space a hard constraint on joint semantic, interaction, and load optimisation. Sender grouping, clustering, graph partitioning, and chronological pinning are established techniques and are not individual novelty claims. The theorem is confined to uncoordinated same-snapshot additive merging; coordinated execution can admit other partitions.

II. Related Work

The independent regional model [2] differs from conflict re-execution [9], [10], static parallelism analysis [11], dependency schedules [12], [13], speculation [14], object-level independence [15], coordinated ordering [7], [8], and cross-shard coordination [16], [17]. Our admissibility result concerns only the uncoordinated barrier model.

SemantiChain couples semantic placement with retrieval [3]; ALPS adds a model-derived admissibility constraint and explicit interaction and load terms.

Workload-aware placement is the closest algorithmic line. OptChain uses an online graph [18]; Shard Scheduler migrates objects [19]; BrokerChain uses broker accounts [20]; TxAllo optimises traffic and balance [4]; and Orbit and Mosaic adapt account placement [5], [6]. Section V-F compares ALPS with the TxAllo allocation objective. Because A-TxAllo already

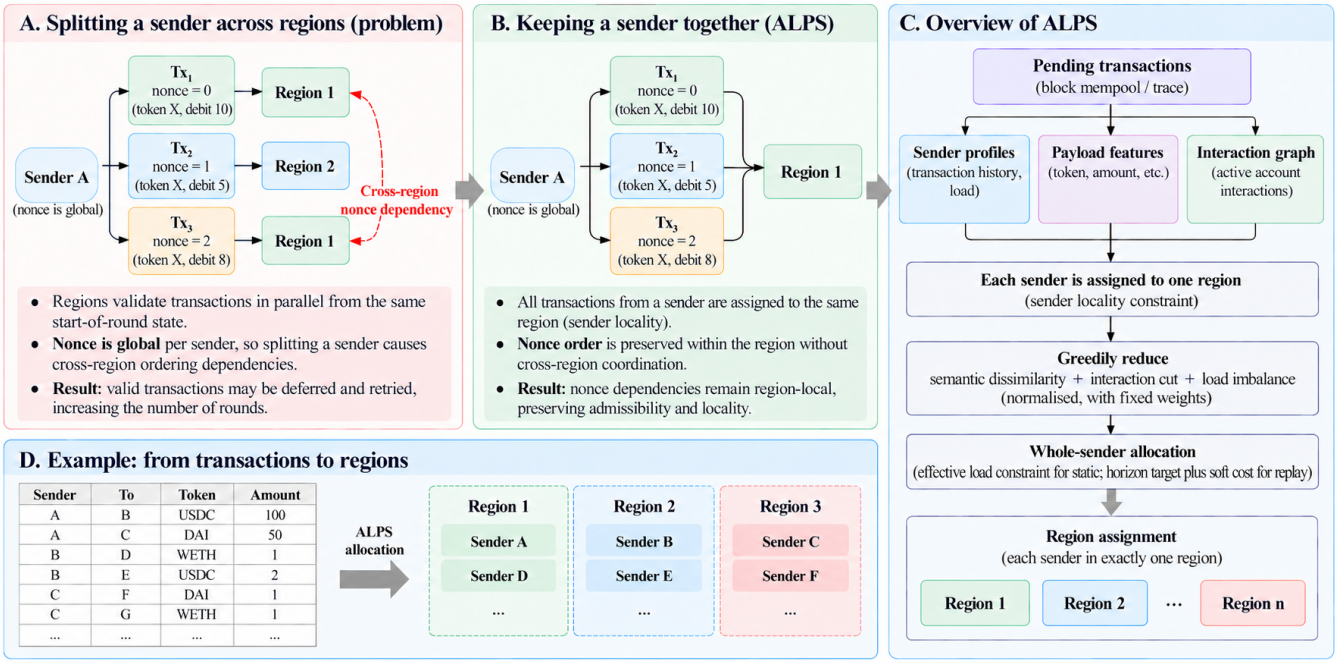


Fig. 1. ALPS overview. Splitting one sender creates cross-region account-nonce dependencies (A); sender-local assignment keeps those dependencies within one region (B). ALPS combines sender profiles, payload features and active-sender interactions subject to whole-sender placement (C), producing a regional allocation such as the illustrative direct ERC-20 mapping in (D).

places new nodes over time, our chronological experiment tests robustness rather than precedence.

Database partitioning provides another useful comparison. Schism balances a co-access graph to reduce distributed transactions [21]. ALPS follows a similar idea, but it moves only whole senders so that the resulting allocation remains admissible without abort or retry coordination. We also apply weighted multilevel METIS [22] to the active-sender graph. This comparison separates a general graph-cut objective from ALPS's semantic term and effective load constraint.

III. Proposed Work

A. System Model and Admissibility

The model has two separate locality requirements. Guarded-key locality protects balances, while sender locality keeps account-nonce dependencies within one region.

1) Independent-barrier Execution: Let T be a transaction set and $P : T \to \{1, \ldots, n\}$ its static region assignment. At the start of round k, every region reads the same committed state σ_k. Each region executes concurrently with the others, processes its transactions sequentially on a private copy, and accepts a debit only if its local balance is sufficient; an insufficient-balance debit is rejected in that execution. Regions neither observe nor serialize with one another during the round. At the barrier, all accepted debit and credit deltas are merged additively to form σ_{k+1}. This follows the execution structure of the base parallel-mining design [2]; proof-of-work is independent of the state argument considered here.

An ERC-20 balance is associated with both an account and a token contract. Let $b = (a, c)$ denote the balance of account a in token contract c. A transfer $t = (a, d, c, x, q)$ conditionally consumes x from debit key $g(t) = (a, c)$, credits (d, c), and carries account-wide nonce q. Credits are unconditional additions, whereas debits are guarded non-commutative operations.

For balance safety, all transactions that debit the same guarded key must stay in the same region. This requirement, called *guarded-key locality*, can be written as

$$g(t_i) = g(t_j) \implies P(t_i) = P(t_j).$$

A partition is *balance-safe barrier-admissible* if, for every non-negative starting balance vector and every same-round transfer workload, independently validated deltas cannot merge into a negative balance. This property concerns balance non-negativity only, not nonce validity, completeness, or liveness.

Theorem 1 (Balance-Safety Admissibility)**.** *Under the concurrent same-snapshot, additive-merge execution just defined, without cross-region serialization or revalidation, a static transfer partition is balance-safe barrier-admissible for every non-negative starting balance vector and same-round workload if and only if it satisfies guarded-key locality.*

Proof. For sufficiency, consider a guarded key b in round k. Guarded-key locality implies that only one region r can debit b. Sequential local validation in that region gives

$$D_r(b) \le \sigma_k(b) + C_r(b),$$

where $D_r(b)$ and $C_r(b)$ are the debit and credit totals of region r. After the regional changes are merged,

$$\sigma_{k+1}(b) = \sigma_k(b) + \sum_j C_j(b) - D_r(b) \geq \sum_{j \neq r} C_j(b) \geq 0.$$

Thus, the merged balance cannot become negative, and the same argument applies inductively over subsequent rounds.

For necessity, suppose the static assignment places two debits of the same guarded key in different regions in the same round, with positive debit totals d_1 and d_2, and no credits are made to that key. If the starting balance is $\max(d_1, d_2)$, both regions can accept their debits because they validate against the same starting state. After the regional changes are merged, the balance becomes

$$\max(d_1, d_2) - d_1 - d_2 < 0.$$

Hence any such partition fails to be balance-safe barrier-admissible for some starting balance and same-round workload. □

The necessity direction is specific to uncoordinated independent barriers. Coordination, revalidation, or re-execution can prevent the simultaneous acceptance used by the counterexample; guarded-key locality remains sufficient in those systems, but need not be necessary.

2) Sender Locality for Global Nonces: Ethereum assigns a transaction nonce to the sending account, not to a token. If one sender is split across regions, a transaction with a later nonce may have to wait for an earlier transaction to commit in another region. ALPS therefore assigns all transactions from sender a, across all tokens, to one region. For the direct calls in our evaluation, the outer signer is also the token debitor. Sender locality is therefore stronger than guarded-key locality for these calls and keeps both token-balance updates and account-nonce order in one region. Calls from one signer that debit different tokens illustrate the distinction: their guarded keys may be separated safely by Theorem 1, while consecutive account nonces still require ordered progress. Router-mediated calls and arbitrary contract read/write sets fall outside this implication.

Corollary 1 (Direct-Call Locality)**.** *For the evaluated direct ERC-20 calls, where the outer signer is the debited token holder, sender locality implies guarded-key locality and makes every submitted account-nonce dependency region-local.*

Proof. All guarded keys (a, c) belonging to sender a map to the sender's single region, and every submitted transaction carrying a's account nonce maps to that same region. □

A nonce-aware mempool can defer a future-nonce transaction and retry it after lower nonces commit, restoring eventual progress at the cost of extra barriers. Section V measures this cost and separately stress-tests guarded-key balance safety.

B. Proposed Method: ALPS

ALPS uses the sender as its allocation unit. It combines semantic locality, interaction locality, and historical-load control. We first describe its static allocation method and then its fixed-horizon chronological extension.

1) Allocation Unit and Objective: ALPS treats each active sender a and its complete transaction sequence as one indivisible unit. Let w_a denote its historical transaction count, x_a its mean payload feature vector, and e_{ab} the symmetrized number of observed submitted calls in either direction between active senders a and b. Define $W = \sum_a w_a$, $M = \sum_{\{a,b\}} e_{ab}$, the mean regional load $\bar{L} = W/n$, and $L_r = \sum_{a:z_a=r} w_a$. The exact dimensionless objective is

$$\begin{aligned} J = \frac{1}{W} \sum_a w_a \frac{\|x_a - \mu_{z_a}\|_2^2}{s} \\ + \frac{\lambda_g}{M} \sum_{\{a,b\}} e_{ab} \mathbf{1}[z_a \neq z_b] \\ + \frac{\lambda_l}{n} \sum_{r=1}^{n} \left(\frac{L_r}{\bar{L}} \right)^2 \end{aligned} \tag{1}$$

The heuristic reduces this objective while keeping each sender in one region and enforcing $L_r \leq C_{\text{eff}}$. A feasible cap cannot be smaller than the largest sender, so the target is $C_0 = \max(\alpha \bar{L}, \max_a w_a)$. If a whole-sender packing satisfies C_0, ALPS uses $C_{\text{eff}} = C_0$. Otherwise, the LPT fallback provides a feasible packing, and the effective cap is raised only to the maximum load of that packing. In every reported run, $\max_a w_a \leq \alpha \bar{L}$ and no relaxation was needed. Thus, $C_0 = C_{\text{eff}} = \alpha \bar{L}$ in these runs.

The scale s is computed once, after the initial weighted k-means fit. It is the median nonzero squared distance from each active-sender profile to each initial prototype. The same value is used during placement, prototype updates, and refinement. The experiments use $\alpha = 1.15$, $\lambda_g = 0.15$, and $\lambda_l = 0.20$. Section V reports the graph-weight frontier; therefore, $\lambda_g = 0.15$ is not presented as a universal optimum. The implementation requires $W > 0$ and at least n active senders. It replaces zero values of M or s with one, keeps the previous prototype for an empty region, and resolves equal-cost choices by selecting the lowest region index.

The best operating point depends on the deployment. Let X be the active-account interaction cut defined in Section IV. A deployment may combine retrieval cost and interaction cost in the score $D_\kappa = R_{90} + \kappa X$. Here, κ expresses the interaction-cut penalty in R_{90}-equivalent units. Under this score, ALPS is preferable to a baseline B when

$$\kappa > \kappa^\star = \frac{R_{90}^{\text{ALPS}} - R_{90}^{B}}{X_B - X_{\text{ALPS}}} \tag{2}$$

provided $X_{\text{ALPS}} < X_B$. A non-positive threshold means that ALPS weakly dominates. This formulation makes the retrieval–interaction trade-off explicit instead of hiding it inside a single composite score.

Algorithm 1 Static ALPS: all assignments and moves keep senders whole.

Input: $T_h, n, \alpha, \lambda_g, \lambda_l$; objective J in (1)

Compute sender profiles (w_a, x_a) and interaction weights e_{ab}
Initialise prototypes by transaction-weighted k-means; $L \leftarrow 0$, $z \leftarrow \emptyset$
Set s from the initial prototypes and hold it fixed
$C_0 \leftarrow \max(\alpha W/n, \max_a w_a)$
for senders a ordered by decreasing w_a, then account ID **do**
    $F \leftarrow \{r : L_r + w_a \le C_0\}$; if empty, $F \leftarrow \{\arg\min_r L_r\}$
    Assign a to the region in F with least incremental J against placed senders; update L
end for
if $\max_r L_r > C_0$ **then**
    Restart packing in the same sender order, assigning each sender to the least-loaded region
end if
$C_{\text{eff}} \leftarrow \max(C_0, \max_r L_r)$
for at most six sweeps **do**
    Recompute transaction-weighted prototypes; retain empty ones
    for senders a in the same order **do**
        Find the feasible destination minimising exact ΔJ; move a and update L only if $\Delta J < -10^{-12}$
    end for
    Stop if no sender moved
end for
return z (ties retain the current region, otherwise lowest region ID)

The objective has three parts. The payload-distance term supports asset-query locality. The graph term penalises interactions between active accounts in different regions. The load term penalises uneven historical load. A separate hard constraint limits the maximum historical load and is distinct from the per-round block capacity B. Sender indivisibility is a hard rule outside the objective, so the optimiser cannot split a sender to reduce the objective.

Algorithm 1 gives the static heuristic. If greedy seeding exceeds C_0, largest-processing-time-first (LPT) packing provides a feasible cap for the refinement stage. No reported run required this relaxation. Proposition 1 shows that refinement preserves feasibility and reduces the objective; it does not show that ALPS finds a global optimum.

a) Static Time Complexity: Let $N = |T_h|$, A be the active senders, E their interaction edges, I the weighted k-means iterations, and $S \le 6$ the refinement sweeps. Profiling costs $O(Nd)$, weighted k-means $O(InAd)$, sender sorting $O(A \log A)$, and the seed or one sweep $O(n(Ad + E))$. Thus worst-case static time is

$$O(Nd + InAd + A \log A + (S+1)n(Ad + E)). \quad (3)$$

LPT adds $O(A \log A + A \log n)$ if invoked; no reported run required it. Allocator storage excluding the input is $O(Ad + E + n)$.

Proposition 1 (Monotone Constrained Refinement)**.** *With fixed prototypes, every accepted ALPS move strictly decreases* (1) *while preserving sender locality and the effective capacity constraint. Replacing each prototype by its transaction-weighted mean cannot increase the semantic term.*

Proof. Each accepted whole-sender move has negative exact ΔJ and a feasible destination, preserving locality and capacity. The positive normalisation s is fixed throughout refinement. For fixed assignments, weighted means minimise the semantic numerator; division by the same s therefore preserves the non-increase, so prototype updates cannot increase J. The six-sweep limit may stop refinement before a one-move local optimum. □

2) Fixed-Horizon Chronological Placement: Senders seen in the history keep their assigned regions. When an unseen sender a first appears in the future stream, ALPS selects a region and pins that sender there for the rest of the replay. The selected region is

$$\begin{aligned} z_a &= \arg\min_{r \in F_t} \{\|x_t - \mu_r\|_2^2 / s_t + \lambda_g e_{tr} + \lambda_l \ell_r^2\}, \\ e_{tr} &= \mathbf{1}[\rho_t \neq \perp \ \wedge \ r \neq \rho_t], \qquad \ell_r = L_r / \max(C_f, 1). \end{aligned} \quad (4)$$

Here x_t uses frozen prefix scaling, μ_r is the frozen transaction-weighted history prototype, and ρ_t is the receiver's region if already assigned, otherwise $\perp$. Both numerator and scale use squared Euclidean distance: $s_t = \text{median}\{\|x_t - \mu_j\|_2^2 : \|x_t - \mu_j\|_2^2 > 10^{-12}\}$, or one if the set is empty. The semantic ratio is therefore dimensionless, matching the implementation. With $C_f = \alpha H_f / n$, $F_t = \{r : L_r + 1 \le C_f\}$, falling back to all regions if empty. Loads start at zero for the suffix, increase after every routed transaction, and ties choose the lowest region ID. The semantic weight is one in the reported runs.

This experiment uses a fixed horizon, with $H_f = |T_f|$ predefined for the replay. Given this horizon, each placement decision uses only the payload and placements observed so far. The capacity check is made only when a sender is first placed. Later activity from that sender can raise the regional load above C_f, so this rule does not provide a lifetime hard-cap guarantee.

a) Chronological Time Complexity: Hash-map routing takes $O(1)$ expected time for a pinned sender and $O(nd)$ for each first placement. For F future transactions and U unseen senders, total time is

$$O(F + Und), \quad (5)$$

and the assignment map grows by $O(U)$ entries.

A-TxAllo already carries allocations forward and places new nodes in a 9:1 chronological split [4]; our four-horizon replay is a robustness check. All replay baselines preserve history assignments and pin unseen senders using their own rules. ALPS's replay replaces the static graph term with the currently placed receiver indicator in (4).

IV. Experimental Setup

A. Traces and Feature Construction

Table I lists three Ethereum mainnet traces covering eight ERC-20 contracts and one held-out BNB Smart Chain (BSC) trace covering a separate set of eight tokens. D1–D3 contain

TABLE I

MEASURED TRACES. BLOCKS/SPAN COUNTS HEIGHTS WITH AT LEAST ONE RETAINED ROW OVER THE REQUESTED INCLUSIVE SPAN, NOT SCAN COMPLETENESS. D2, D3, AND C1 SCANNED EVERY REQUESTED BLOCK; D1 HAS NO COMPLETE SCANNED-BLOCK LEDGER. TOP TOKEN IS THE SHARE OF THE MOST FREQUENT CONTRACT; MULTI-TOK. IS THE SHARE OF CALLS/EVENTS FROM REPEAT SENDERS ACTIVE IN MULTIPLE TOKENS.

Trace	Chain	Calls/events	Blocks/span	Senders	Top token	Multi-tok.
Direct calls (D1)	ETH	34,326	513/714	18,747	69.7%	19.5%
Prior-day (D2)	ETH	39,425	713/714	20,719	70.0%	20.9%
Prior-week (D3)	ETH	45,893	499/500	27,963	72.7%	19.9%
Cross-chain (C1)	BSC	25,728	2489/2500	14,082	94.9%	10.3%
Transfer logs	ETH	17,445	81/137	7,195	—	—

only direct `transfer(address,uint256)` calls. For these calls, the outer signer is both the debited token owner and the account-nonce owner. We convert observed signer nonces to ranks within each sender; this keeps their order unchanged. Because other account transactions are not included, these traces represent the collected direct-transfer stream, not the full Ethereum mempool. The traces cover non-overlapping periods on 14, 13, and 7 August 2026. D3 is approximately one week before the other two Ethereum traces. The D2, D3, and C1 manifests record complete scans of their requested inclusive spans. The original D1 acquisition did not retain a complete scanned-block ledger, so its scan coverage cannot be established from the artifact.

The selector, calldata, and nonce fields have the same meaning on both chains. C1 is a difficult case for semantic placement: one token accounts for 94.9% of submitted calls, compared with 69.7–72.7% on Ethereum. A separate `Transfer`-log trace adds more balance events, but it does not identify the outer signer for router-mediated calls. We therefore do not use this trace for nonce-related claims. The artifact manifests list the block ranges, chain IDs, contract addresses, filters, hashes, RPC endpoints, and data provenance.

We collect the traces through public JSON-RPC endpoints. For D2, D3, and C1, the complete-scan collectors request every height in the stated span, store transaction hashes, and abort if any block response is missing. Their manifests separately record the number scanned. The Blocks/span numerator in Table I counts only heights containing at least one retained row; therefore 713/714, 499/500, and 2489/2500 do not indicate missing block responses. D1 lacks that complete ledger and is reported only by its 513 represented heights. We exclude eleven self-transfers from the log trace, leaving 17,445 events (Table I). For the direct-call traces, we keep calls to the listed token contracts that use the standard selector, have nonzero decodable calldata, and name a recipient other than the sender. We exclude router and proxy calls, and remove duplicate sender nonces during loading.

The main experiments do not filter calls by receipt status. They therefore include submitted calls that may later have reverted. In this paper, a "call" does not mean that a token state change succeeded. A reverted submission may still use a nonce and affect the locality metrics.

Raw ERC-20 values cannot be compared directly across tokens because token decimals and price scales differ. For transaction t with raw value v_t and token c, let $\rho_t = \mathrm{clip}(v_t/\operatorname{median}_{j:c_j=c} v_j, 0.1, 20)$. The exact clustering feature is $x_t = [4e_c; \log(1+\rho_t)]$, where e_c is the token one-hot vector; a sender profile is the arithmetic mean of its transactions' x_t vectors. The balance stress model uses transfer amount $8\rho_t$. We do not add Gaussian noise or a synthetic clustering coordinate. Thus, when transaction k-means splits a dominant token, the split is based on an observed difference in transaction amounts.

For static allocation, we calculate token medians from the full allocation trace. For each fixed-horizon temporal split, we calculate the medians from the first 50% of the trace and keep them fixed for both history and suffix transactions. No suffix amount is used to scale temporal features. This common prefix scale keeps feature dimensions comparable across the reported 50–80% history fractions. The temporal horizon is a planning assumption and is varied around the observed suffix length in a separate sensitivity test. A production streaming allocator would need to update these statistics online and choose its planning horizon independently.

B. Schemes and Execution

The main static evaluation compares ADDR-MOD, TX-KMEANS, the sender-based k-means variants, and ALPS. It does not include METIS or TxAllo; these methods are compared separately in Section V-F. ADDR-MOD assigns a sender using $a \bmod n$. TX-KMEANS clusters individual transactions, while the sender-based methods cluster the mean features of each active sender. S-KM-BAL also applies the $1.15\times$ historical-load target. ALPS uses the joint objective described in Section III-B.

We test $n \in \{2, 3, 4, 6, 8, 11\}$ regions and set the capacity of each region to 250 transactions. In the global-nonce experiment, each (account, token) debit key starts with its total observed outgoing amount plus a cushion of one normalised balance unit. This allows sequential execution and any correct retry schedule to commit the full trace. The extra rounds therefore measure ordering and partitioning effects, not failures caused by insufficient starting balances. Each regional mempool selects ready sender/nonce entries, commits up to its capacity, keeps future-nonce transactions, and retries them after the barrier.

The full static sweep uses deterministic seed 0 and k-means `n_init=10`. The temporal, frontier, ablation, and TxAllo comparisons use three initialisations. We base allocation-sensitive conclusions on these sensitivity runs as well as on the exact static audit. The capacity and allocator-runtime audits use five fresh initialisations.

For the balance-safety experiment, we give each observed (account, token) key the same starting balance. We test seven values: $\{1000, 600, 400, 250, 150, 100, 60\}$. We ignore account nonces only in this experiment so that we can test simultaneous debits of one guarded key. A negative balance after the synthetic merge is an exact witness of a balance violation in that run. It is not a reconstruction of historical execution. Balances from different token contracts are never combined.

The state and scheduling experiments set proof-of-work difficulty to zero. We report barrier rounds and the schedule-

level commit rate $Q_s = N_c/R$, where N_c is the number of committed transactions and R is the number of rounds. This measure is not wall-clock transactions per second (TPS). Given regional capacity B, the ratio $Q_s/(nB)$ measures schedule utilisation. It helps isolate dependency stalls from hashing, storage, networking, and consensus. The simulator also has a SHA-256 path for separate benchmarking, but we make no wall-clock claim based on it.

C. Metrics and Temporal Protocol

We report split senders and guarded keys, the share of transactions issued by split senders, barrier rounds, deferred nonce checks, maximum-to-mean regional load, and the active-account interaction cut. For an allocation, let T_{act} contain the submitted calls whose receiver also appears as an active sender. The interaction cut is

$$X = \frac{|\{t \in T_{\mathrm{act}} : z_{\mathrm{sender}(t)} \neq z_{\mathrm{receiver}(t)}\}|}{|T_{\mathrm{act}}|},$$

Interaction coverage is $|T_{\mathrm{act}}|/|T|$. Receiver-only addresses are not included in the cut and are not automatically counted as cross-region. We report regional load as $\max_r L_r/\bar{L}$. To measure retrieval locality, we rank regions for each token by the number of calls or events for that token. R_{90} is the number of top-ranked regions needed to cover at least 90% of the token's calls or events, averaged across the evaluated tokens. A lower R_{90} means that fewer regions need to be scanned; it does not mean that measured query latency is lower. We also run an in-memory benchmark. It stores each region as a category array and tests three types of query. *Oracle-90* ranks regions using the evaluated records and scans the minimum prefix covering at least 90%; it is an approximate, in-sample diagnostic and intentionally omits the remaining records. *History-90* learns each token's ranking and prefix length on the first 70% of the chronological trace, freezes that plan, and reports achieved recall on the final 30%. *Complete* scans all regions and returns every match. Allocations remain the static full-trace allocations used in the head-to-head comparison, so History-90 tests the query planner, not out-of-sample allocator fitting. After warm-up, 250 repetitions per token record contacted regions, examined rows, result recall, median latency, and latency interquartile range. This benchmark measures allocation-related scan work. It does not model an index, network delay, or a distributed storage engine. History-90 excludes tokens that do not appear in the history: there are none on D1–D3, and C1's BUSD token has one suffix call. Each method and seed therefore covers eight/eight/eight/seven token categories. There are no categories that appear only in the history.

a) Prospective Allocation and Queries: We also run a separate 70/30 chronological holdout. Token medians, vocabulary, sender profiles, the interaction graph, allocation, prototypes, and query plans are fitted using the history only. Address IDs are assigned when they first appear. We set the future horizon to $H_f = \lceil 0.5|T_h| \rceil$ before evaluation; it is not chosen from the suffix size. ALPS assigns new senders using (4), while S-KM-BAL does not use the receiver term. Our causal versions of METIS and TxAllo use $0.15e_{tr} + 0.20\ell_r^2$, with the same candidate set and sender-pinning rule. These are our adaptations, not the original streaming protocols.

Unseen tokens use a fixed out-of-vocabulary feature coordinate and the frozen history-wide median raw amount as a fallback scale. This is a heuristic; the fallback does not give amounts from different tokens a common economic meaning. Queries for unseen tokens scan all regions, so all suffix tokens are included. Six regions, three seeds, and seven preset coverage targets produce 48 allocations and 2,688 token-level query measurements. For the matched-recall comparison, we first average tokens within each trace, method, and seed. We then linearly interpolate rows scanned and regions contacted at suffix recalls of $\{.80, .85, .90, .95\}$ between adjacent preset plans, and average the 12 configurations. This interpolation is descriptive: suffix results do not select a coverage target or a deployed plan. Per-trace seed results are provided in the artifact.

Static interaction coverage depends only on the trace and is therefore strategy-invariant: 53.8% for D1, 56.6% for D2, 60.6% for D3, and 45.7% for C1. This conditional metric matches ALPS's active-sender graph while keeping the denominator explicit.

The fixed-horizon replay uses $n = 6$, history fractions $\{0.5, 0.6, 0.7, 0.8\}$, and three k-means initialisations. This gives 12 history/future configurations for each method. The figures show the mean and standard deviation across these configurations. They are sensitivity summaries, not independent samples from the chains. Across the four traces, 60.0% of future transactions come from senders that are not in the training prefix. This makes the treatment of new senders important. At a history fraction of 0.7, we also test four ALPS horizons: $H_f/|T_f| \in \{0.5, 1, 1.5, 2\}$, on every direct-call trace and seed.

V. Performance Evaluation

We evaluate balance safety, retry cost, locality, temporal robustness, component effects, and the comparisons with graph partitioning and TxAllo. The main parameters are $\lambda_g = 0.15$ and $\alpha = 1.15$. We select them using D1 as the development trace and then keep them fixed. D2, D3, and C1 provide checks on temporally separate traces. However, each static allocation is fitted and evaluated on the same trace, so these tests are not out-of-sample allocation tests. The static and head-to-head evaluations use all six region counts. The stress, temporal, and mechanism analyses use $n = 6$. The operating frontier varies only λ_g, and each ablation removes one component. The static sweep contains 120 trace/region/scheme measurements. The balance-stress experiment contains 175 measurements: 140 sender-local and 35 TX-KMEANS.

A. Admissibility under Payload Partitioning

TX-KMEANS splits senders on every trace and, when the number of regions is larger, it also splits guarded keys. Across the 96 sender-local measurements (ADDR-MOD, both sender k-means methods, and ALPS over four traces and six region counts), there are no split senders, no split guarded keys, and no overdrawn accounts. Every method commits the full trace.

TABLE II

RESULTS AT $n = 6$ FOR ALL DIRECT-CALL TRACES. SPLIT KEYS ARE GUARDED (ACCOUNT, TOKEN) KEYS. AN OVERDRAWN ACCOUNT IS A UNIQUE ACCOUNT WITH AT LEAST ONE NEGATIVE GUARDED KEY; RANGES SPAN THE SEVEN-LEVEL BALANCE STRESS. LOWER IS BETTER EXCEPT Q_s.

Method	Split keys	Rounds	Q_s	Overdrawn accts.	R_{90}	Load	Active cut (fraction)
D1 ETH 08-14							
ADDR-MOD	0	33	1,040	0	5.88	1.43	.837
TX-KMEANS	1,021	1,068	32	19–48	1.38	2.27	.425
S-KM-BAL	0	27	1,271	0	2.25	1.15	.646
ALPS	0	27	1,271	0	3.00	1.15	.260
D2 ETH 08-13							
ADDR-MOD	0	37	1,066	0	5.88	1.39	.841
TX-KMEANS	1,061	1,388	28	20–48	1.38	2.28	.432
S-KM-BAL	0	31	1,272	0	2.38	1.15	.670
ALPS	0	31	1,272	0	2.88	1.15	.279
D3 ETH 08-07							
ADDR-MOD	0	38	1,208	0	5.50	1.22	.834
TX-KMEANS	1,067	843	54	12–36	1.38	2.08	.614
S-KM-BAL	0	36	1,275	0	3.12	1.15	.818
ALPS	0	36	1,275	0	2.88	1.15	.528
C1 BSC							
ADDR-MOD	0	21	1,225	0	4.62	1.21	.837
TX-KMEANS	478	448	57	13–34	1.25	2.93	.413
S-KM-BAL	0	20	1,286	0	2.62	1.15	.859
ALPS	0	19	1,354	0	2.00	1.07	.236

By contrast, TX-KMEANS splits 66–1,592 senders and up to 1,334 guarded keys. Across the four traces and region counts, transactions from split senders make up 10.3–44.4% of each trace (Fig. 2(a)). Table II shows the results for $n = 6$.

The difference between guarded-key locality and sender locality is clear when $n = 2$. Across all four traces, TX-KMEANS does not split guarded keys, but it still splits 66–551 senders. With two regions, clustering places each token's transactions on one side. As a result, every $(\text{account}, \text{token})$ balance stays in one region, so balance safety is preserved. However, an account that uses several tokens can still be split between regions. This creates cross-region dependencies for its account-wide nonce. The result shows why guarded-key locality alone does not ensure sender locality.

The balance-stress experiment gives further evidence. It contains 175 measurements: five traces, seven starting balances, and five methods. In each of the 35 TX-KMEANS cases, 6–48 unique accounts are overdrawn. All 140 sender-local cases have zero overdrawn accounts. The results at the lowest starting balances are not monotone. With a smaller balance, a regional transfer may fail before two conflicting debits are both accepted and merged. The number of balance-safety witnesses therefore does not have to increase as the starting balance decreases.

B. Retry Cost of Sender Fragmentation

Deferring and retrying future-nonce transactions allows the full trace to commit, but it does not remove the scheduling cost. After retries, every method commits 100% of each trace with no pending transactions. However, TX-KMEANS needs 156–1,760 rounds, compared with 11–106 rounds for ALPS. It also performs up to 3.13 million future-nonce checks. Its schedule-level commit rate, Q_s, falls to 22–253 transactions per round, compared with 432–2,338 for ALPS.

At $n = 6$, TX-KMEANS needs about $40\times$, $45\times$, $24\times$, and $24\times$ as many barrier rounds as ALPS on D1, D2, D3, and C1, respectively (Fig. 2(b), Table II). This comparison measures the scheduling cost of splitting senders whose transactions have an account- nonce order. It is not a speedup over the strongest admissible method: ALPS and balanced sender k-means both need 27 rounds on D1. The claim for ALPS is instead that, among sender-local methods, it offers a different balance between retrieval locality and interaction locality under the same load constraint.

C. Locality and Load Trade-offs

Compared with ADDR-MOD, ALPS improves both locality measures in all 24 trace/region settings, while keeping each sender in one region. The additional cost is allocator computation (Section V-E). R_{90} decreases by 18.8–68.5% (mean 45.9%), and the interaction cut decreases by 35.2–75.9% (mean 57.7%). The static load does not exceed the 1.15 target in any reported run.

A more useful comparison is with balanced sender k-means, which also satisfies the locality requirements. ALPS reduces the interaction cut in all 24 settings by 12.8–82.5% (mean 50.9%). Its retrieval measure, R_{90}, improves in 19 settings, is unchanged in two, and is worse in three. The three cases occur at $n = 6$ on D1 and D2 and at $n = 4$ on D3. Only the D1 case uses the development trace for parameter selection. We report these cases to show that ALPS does not improve every measure in every setting.

In most settings, ALPS improves both measures. In the other settings, the break-even ratio $\kappa^\star$ shows when the simpler method may be a better choice under the score in (2). For D1 with $n = 6$, $\kappa^\star = 1.94$. A value above this threshold gives more weight to interaction cut and favours ALPS; a lower value favours SENDER-KMEANS-BAL. At larger region counts, ALPS often weakly dominates, so the threshold is non-positive.

Figure 3 helps explain why the three retrieval exceptions occur at smaller values of n. As the number of regions increases, the retrieval cost of ADDR-MOD rises. This gives semantic partitioning more room to improve R_{90}. At $n = 11$, ALPS lowers R_{90} by 10.0–37.8% compared with balanced sender k-means across all four traces. Unconstrained SENDER-KMEANS also gives strong semantic locality, but its load reaches 4.45 at $n = 11$. The capacity-aware methods remain at or below 1.15.

D. Fixed-Horizon Replay Robustness

Compared with balanced sender k-means, ALPS lowers the conditional future interaction cut for transactions whose receiver is already placed. The reductions are 19.3%, 23.7%, 37.9%, and 14.1% on D1, D2, D3, and C1, respectively. ALPS also has a lower observed load. The retrieval results are mixed: R_{90} is 4.8% and 8.0% higher on D1 and D2, is almost unchanged on D3, and is 30.3% lower on C1. Unconstrained SENDER-KMEANS has lower interaction cuts on D3 and C1 (.327 and .576, compared with .513 and .745 for ALPS), but its load is

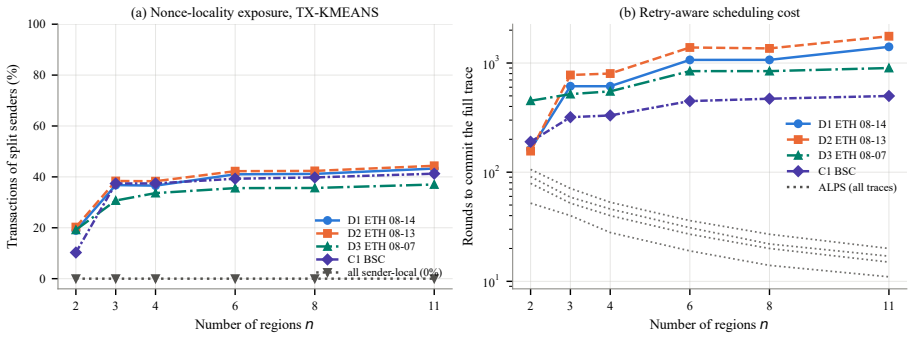


Fig. 2. Locality violations and retry-aware scheduling cost across the four direct-call traces. Sender-local schemes have zero split-sender exposure in (a), while (b) compares TX-KMEANS and ALPS round counts.

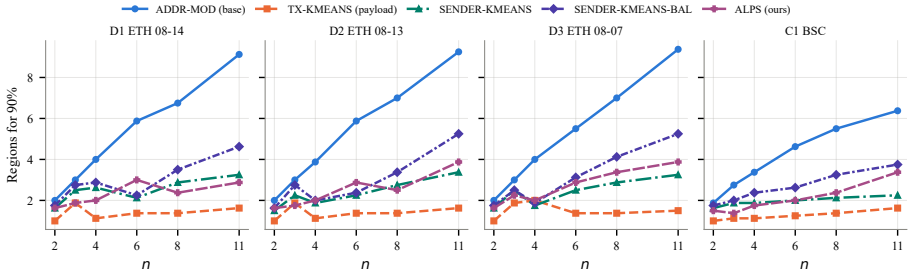


Fig. 3. Semantic retrieval cost across all direct-call traces. TX-KMEANS is an infeasible, capacity-unconstrained comparator that violates sender locality; the other methods are sender-local.

TABLE III
DECLARED-HORIZON SENSITIVITY AT $n = 6$ AND HISTORY FRACTION 0.7. MEANS COVER FOUR TRACES AND THREE INITIALISATIONS; LOWER IS BETTER.

$H_f/\lvert T_f\rvert$	R_{90}	Active cut	Max/mean load
0.5	2.802	.656	1.260
1.0	2.573	.615	1.426
1.5	2.573	.581	1.724
2.0	2.573	.552	1.858

much higher: 3.68 and 2.58, compared with 1.48 and 1.43 for ALPS.

The causal interaction coverage is 26.4–44.3%: the conditional cut is defined only for calls whose receivers have already been placed when they arrive. Sender pinning preserves sender locality, but later activity can raise load above the 1.15 target used at first placement. Per-trace results are provided in the artifact.

Table III reports horizon sensitivity across four traces and three initialisations. Shorter horizons give lower load, but higher interaction cut and retrieval cost. Longer horizons reduce the cut, but increase load. Thus, H_f controls a trade-off between interaction locality and load.

E. Component Analysis and Allocator Overhead

Removing the semantic term raises R_{90} to 5.00/5.25/4.88/4.12 on D1/D2/D3/C1, while it lowers the interaction cut. Removing the capacity constraint raises load to 1.28–1.52, above the measured 1.15 maximum-imbalance target. Setting $\lambda_g = 0$ removes the interaction term. This variant is not identical to balanced sender k-means: it uses transaction-weighted prototypes and exact objective refinement.

The static graph-weight curves may flatten or reverse above 0.35. This is consistent with the non-convex search. We do not claim one universal best weight.

A five-seed audit tests $\alpha \in \{1.05, 1.10, 1.15, 1.25, 1.50\}$ with six regions on all four traces. No run raises its requested cap. Increasing α from 1.05 to 1.15 lowers the cut on every trace. Larger values produce different, non-monotone changes across traces. The error intervals in the artifact measure variation across initialisations, not variation across workloads.

Allocator computation is noticeable at this scale. For D1 prefixes of 5,000, 10,000, 20,000, and 34,326 calls, fresh single-worker runs over three seeds take median profile-and-allocation times of 2.50, 5.32, 8.08, and 12.60 s on an Intel i9-14900. On the full trace, which has 18,747 active senders and 16,120 interaction edges, the peak process working set is 248.4 MiB. No run raises the capacity bound. These measurements describe allocator overhead; they are not production TPS measurements.

A separate comparison uses five seeds and fresh processes with six regions. The trace-wise median times are < 0.01 s for ADDR-MOD, 0.16–0.26 s for TX-KMEANS, 0.29–0.68 s for S-KM-BAL, 0.13–0.28 s for METIS, 4.17–6.88 s for TxAllo, and 10.99–15.74 s for ALPS. CSV loading is excluded. The times include the profile or graph construction required by each method.

F. Comparison with Graph Partitioning and TxAllo

Sender-weighted METIS produces no split senders in all 72 runs. Across the 24 trace/region settings, ALPS has an R_{90} that is 18.2–64.4% lower than METIS (mean 39.4%). METIS, in turn, has an interaction cut that is 32.9–91.1% lower than ALPS (mean 63.4%), and its maximum load is 1.058. This result shows the difference between the two objectives. METIS uses the account-interaction graph, while R_{90} measures how token activity is spread across regions.

The TxAllo comparator [4] is a reimplementation of its allocation objective. It uses Louvain initialisation, candidate-community moves, and coordinate descent on the throughput score. For each shard r, workload σ_r is the local transaction count plus $(\eta-1)$ times the weight of incident cross-shard edges. An account moves only to a neighbouring community's shard when the move strictly lowers $\max_r \sigma_r$. Ties keep the current assignment. We set $\eta = 2$ and stop after eight sweeps or after a sweep with no accepted move. The results are means over three seeds and eight trace/region settings. We exclude consensus, settlement, brokers, and A-TxAllo epoch updates. This is a comparison of allocation objectives under one execution model, not a comparison of deployed throughput.

The reimplementation assigns each account to one shard and records no split senders or guarded keys in the eight trace/region settings. It therefore also meets the locality condition in Section III-A. The two methods, however, favour different objectives. Compared with ALPS, the reimplemented objective lowers the active-account cut in all eight settings by 16.7–70.2% (mean 45.0%). ALPS lowers R_{90} in all eight settings by 38.1–60.8% (mean 46.0%). These percentages are relative changes computed from the unrounded three-seed mean of each metric in each trace/region setting; they are not averages of seed-wise percentage changes. The reimplementation gives more weight to cross-region interactions. ALPS also includes a semantic-distance term to improve retrieval locality, which we measure using R_{90}. At $n = 11$, the reimplementation allows loads of 1.16–1.67. Static ALPS keeps its effective load at or below 1.15.

We use a simple cost model to describe when the retrieval benefit may justify the higher interaction cut. For one submitted transaction, let q be the number of token queries drawn uniformly across the evaluated categories, as in R_{90}. Let c_q be the cost of scanning one region, c_x the cost of one cross-region active interaction, and p the fraction of calls eligible for the interaction measure. The expected cost, in common units, is

$$C = qc_qR_{90} + pc_xX.$$

Under this model, ALPS is cheaper than METIS when $qc_q/c_x > p(X_{\mathrm{ALPS}} - X_{\mathrm{METIS}})/(R_{90}^{\mathrm{METIS}} - R_{90}^{\mathrm{ALPS}})$. Using the measured $p = 0.457$–0.606, the threshold is 0.020–0.081 across Table IV. The corresponding threshold against TxAllo is 0.011–0.119. These thresholds cover operating costs before allocator computation. A deployment with a different query mix should replace R_{90} with the weighted mean for that mix. The model therefore makes clear when retrieval locality may justify a higher interaction cut; it does not claim that ALPS is always better.

The model can also include the cost of rebuilding an allocation. Let A_m be the cost of one rebuild by allocator m, measured in the same units as c_q and c_x. Let N be the number of submitted transactions and $Q = qN$ the number of token queries before the next rebuild. Then

$$\bar{C}_m = A_m/N + qc_qR_m + pc_xX_m. \tag{6}$$

Let $\Delta A = A_{\mathrm{ALPS}} - A_B$ and $\Delta X = X_{\mathrm{ALPS}} - X_B$. ALPS is cheaper than baseline B only when $Qc_q(R_B - R_{\mathrm{ALPS}}) > \Delta A + Npc_x\Delta X$. When $R_B > R_{\mathrm{ALPS}}$, the rebuild cost adds $\Delta A/[Nc_x(R_B - R_{\mathrm{ALPS}})]$ to the earlier threshold. More frequent rebuilding increases this cost. If the rebuild interval is τ and the transaction rate is v, then $N = v\tau$. This model assumes that query and interaction statistics remain stable during that interval; we do not test that assumption.

In the five-seed, fresh-process, full-trace allocator measurements reported in Section V-E, ALPS takes 10.87–15.45 s more than METIS to allocate the regions. If we assume an equal-recall query saving of 1 μs, 10.87–15.45 million queries would be needed to recover this extra computation alone. This is an illustrative calculation, not a deployment measurement. A full comparison must also include shared resource costs, interaction costs, migration, and index rebuilding. Complete queries scan all regions and rows, so these experiments do not show an allocation-related scan saving for complete queries.

Table V reports the query scan results. Oracle-90 gives ALPS 96.2% recall, so it is an approximate result, not a complete query. Under History-90, ALPS reaches 91.4% recall while scanning 2.74 regions and 4,873 rows. Compared with METIS and TxAllo, it scans 38.8–39.7% fewer regions and 41.8% fewer rows. Its recall is also 2.9–4.1 percentage points higher. S-KM-BAL contacts 8.6% fewer regions than ALPS, but ALPS scans 15.1% fewer rows and has 7.8 percentage points higher recall.

A complete query scans all six regions and returns every match. Its mean per-token median latency is 7.93–8.87 μs. History-90 reports the median and interquartile range over 250 in-memory repetitions; these timings establish neither distributed nor equal-recall latency gains.

Figure 4 shows the trade-off between recall and scan work for the preset training coverage targets $\{0.50, 0.70, 0.80, 0.90, 0.95, 0.99, 1.00\}$. Each region ranking and prefix length is fixed using history counts. The suffix is used only to measure the recall achieved by that plan. The curves allow comparison over the recall ranges shared by the methods. However, the connected points are descriptive: they are not plans selected to reach the same suffix recall. The 1.00 control contacts all six regions. Allocations are retrospective; the prospective holdout follows.

A one-factor sensitivity analysis at $n = 6$ varies $\eta \in \{1.25, 1.5, 2, 3, 4\}$ with eight sweeps and varies the sweep cap over $\{4, 8, 16, 32\}$ with $\eta = 2$, across four traces and three seeds. ALPS retains lower R_{90} in all 12 trace/seed configurations for every value of η. The reimplementation retains the

TABLE IV

HEAD-TO-HEAD ALLOCATION RESULTS AT THE TWO REPORTED REGION COUNTS. ENTRIES ARE THREE-SEED MEANS. R IS R_{90}, X IS ACTIVE-ACCOUNT INTERACTION CUT, AND L IS MAXIMUM-TO-MEAN LOAD; LOWER IS BETTER.

		ALPS			METIS-SENDER			TxAllo		
Trace	n	R	X	L	R	X	L	R	X	L
D1	6	3.04	.263	1.150	4.67	.083	1.000	4.92	.119	1.010
D1	11	2.88	.314	1.135	8.08	.122	1.030	7.33	.166	1.156
D2	6	2.96	.246	1.123	4.62	.092	1.000	5.00	.125	1.008
D2	11	3.75	.320	1.147	7.75	.134	1.030	7.50	.186	1.206
D3	6	2.88	.524	1.150	5.04	.235	1.001	4.75	.156	1.106
D3	11	3.83	.444	1.150	7.62	.295	1.058	7.75	.211	1.665
C1	6	2.00	.236	1.073	3.88	.152	1.000	3.62	.197	1.008
C1	11	3.25	.349	1.150	5.62	.215	1.049	5.71	.253	1.177

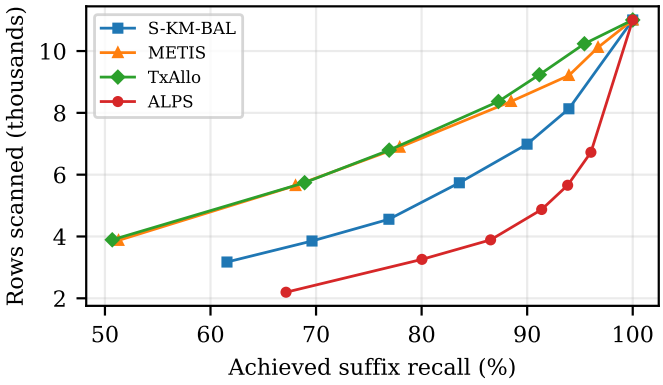


Fig. 4. Frozen historical query-plan sweep: mean suffix recall versus mean rows scanned over the same eligible token/seed population as Table V. Each marker is a preset training coverage target; lines join targets in order. Allocations are fitted retrospectively on the full trace.

TABLE V

SIX-REGION IN-MEMORY TOKEN-QUERY BENCHMARK OVER FOUR TRACES AND THREE ALLOCATION SEEDS. AGGREGATE COLUMNS AVERAGE TOKENS; HISTORY-90 PLANS ARE FROZEN FROM THE FIRST 70% OF EACH TRACE. LATENCY IS MEAN PER-TOKEN MEDIAN WITH MEAN REPETITION IQR IN PARENTHESES, IN μS.

	Oracle-90		History-90		
Method	Reg.	Rows	Reg.	Rows	Recall
S-KM-BAL	2.68	17,604	2.51	5,738	.836
METIS	4.55	28,067	4.48	8,369	.885
TxAllo	4.57	28,088	4.55	8,368	.873
ALPS	2.72	15,991	2.74	4,873	.914

History-90	C1	D3	D2	D1
S-KM-BAL	3.05 (.11)	3.85 (.10)	3.03 (.12)	3.18 (.08)
METIS	5.51 (.09)	6.63 (.13)	5.91 (.08)	6.24 (.29)
TxAllo	4.64 (.11)	5.49 (.12)	5.60 (.15)	5.37 (.10)
ALPS	2.60 (.09)	3.62 (.08)	3.65 (.09)	3.73 (.08)

lower interaction cut in all 12 trace/seed configurations through $\eta = 3$ and in 10 of 12 configurations at $\eta = 4$. It exceeds the 1.15 load target once at each of $\eta = 3$ and $\eta = 4$. All four sweep caps produce identical metrics, indicating convergence within four sweeps in these settings.

G. Prospective Allocation and Receipt Checks

Table VI reports the history-only method with a preset 90% coverage target. ALPS reaches 90.78% mean suffix recall and scans 4,404 rows. The comparators reach 79.10–88.46% recall and scan 5,338–8,344 rows. All 48 allocations keep each sender in one region. The complete-query and cold-token controls return all matches. The causal interaction coverage is 34.79%. ALPS has 6.1 times the interaction cut of METIS+C (.592 versus .097) and a higher maximum-to-mean load (1.534 versus 1.191).

TABLE VI

PROSPECTIVE ALLOCATION AND QUERIES: 70/30 HOLDOUT, SIX REGIONS, THREE SEEDS, 90% HISTORICAL COVERAGE TARGET. QUERY COLUMNS AVERAGE ALL SUFFIX TOKENS/SEEDS; X AND LOAD AVERAGE 12 TRACE/SEED CONFIGURATIONS PER METHOD. +C DENOTES OUR CAUSAL EXTENSION.

Method	Recall	Regions	Rows	X	Load
S-KM-BAL	.7910	2.95	5,338	.773	1.671
METIS+C	.8846	4.58	8,312	.097	1.191
TxAllo+C	.8467	4.63	8,344	.092	1.179
ALPS	.9078	2.82	4,404	.592	1.534

ALPS therefore favours retrieval at the expense of interaction locality and load. Chronological placement has no lifetime load cap. Because local timings used different Python environments and overlapping runs, we compare scan work rather than latency.

Figure 5 compares scan work at a common achieved recall. At 90% recall, interpolation gives 4,419 rows and 2.94 regions for ALPS. The corresponding estimates are 7,139 rows and 3.97 regions for S-KM-BAL, 8,459 rows and 4.71 regions for METIS+C, and 9,020 rows and 5.06 regions for TxAllo+C. These are descriptive interpolations between frozen plans, rather than executed equal-recall plans or latency measurements. The 4,404 rows in Table VI directly average the preset 90%-coverage plan, whose mean recall is 90.78%; Figure 5 first interpolates each trace/method/seed configuration to 90% achieved recall and then averages 12 estimates. Because interpolation and averaging do not commute, its 4,419-row estimate is not a monotonicity comparison with 4,404.

With an ideal per-region inverted index, each returned match would require one posting read. At equal recall, this would not reduce the number of postings read because of the allocation. Placement could still reduce the number of regions contacted. Figure 5 therefore provides evidence about unindexed scan work and region fanout, not about indexed databases or end-

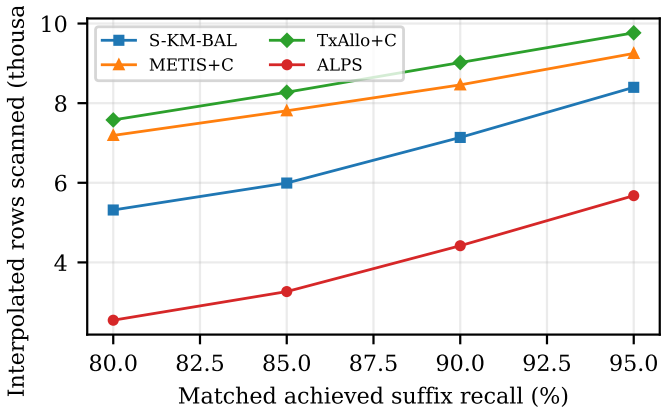


Fig. 5. Prospective matched-recall scan work. Each point linearly interpolates between adjacent plans whose coverage targets were fixed from history before the suffix; lines are descriptive and do not select a suffix-tuned plan.

to-end query speed.

The matched-90% results also allow an example of rebuild amortisation against METIS+C. ALPS contacts 1.77 fewer regions, but its conditional cut is .495 higher at coverage $p = .348$. In the 70/30 prospective experiment underlying Table VI, the timed operation is the history-prefix static allocator alone; future routing and query execution are excluded. Averaged over four traces and three seeds, it takes 9.36 s for ALPS and 0.124 s for METIS+C, a 9.24 s difference. Under the cost model in (6), assume $q = .10$ queries per transaction, a regional query cost of 1 ms, and an active- interaction cost of .1 ms. The estimated break-even point is about 5.8×10^4 transactions. If the interaction cost is 1 ms, it rises to 1.87×10^6 transactions. If $q = .01$ and the interaction cost remains .1 ms, it rises to 1.87×10^7 transactions. These are example scenarios, not measured service costs. If query savings per transaction do not exceed the interaction penalty, the extra allocation cost is not recovered by this model.

The prospective sensitivity test varies $\lambda_g \in \{.15, .60, 2.40\}$ in both history fitting and causal placement. The protocol and query targets stay fixed across four traces and three seeds; no parameter is selected using suffix results. At the 90% target, the mean cuts are .592, .434, and .179. The corresponding recalls are 90.78%, 91.45%, and 89.26%; scanned rows are 4,404, 5,262, and 6,564; and loads are 1.534, 1.614, and 1.551. The 0.67 percentage-point increase in mean recall at $\lambda_g = .60$ is descriptive and is not a significance claim.

The artifact reports variation across seeds within each trace separately from variation across the four different traces. The pooled standard deviation across 12 configurations is only a descriptive summary. A clean-environment rerun reproduced the scientific fields in Tables II–VI and Figures 2–4. The packaged deterministic analysis also regenerates Figure 5; timing results were excluded from this check. A higher graph weight reduces interaction cut but increases scan work and does not reliably improve load.

For receipt validation, we check a uniform sample of 100 C1 calls from seed 0. The sample confirms successful `Transfer` logs, including in offline replay. Ethereum receipt validation remains unresolved. The supplementary artifact records the attempts and their classifications. A missing receipt does not prove that a transaction reverted. This sample does not validate the Ethereum workload or reconstruct historical balances.

VI. Conclusion and Future Work

Guarded-key locality exactly characterises balance-safe partitions in the uncoordinated same-snapshot additive-merge model; sender locality additionally keeps nonce dependencies local for the evaluated direct calls. ALPS uses these constraints to combine semantic, interaction, and load objectives, meeting its static load target in every reported run.

Empirically, ALPS improves retrieval scan work but trades interaction locality and prospective load against graph-oriented methods. It is therefore suitable when selective-query savings outweigh cut, imbalance, and rebuild costs; the results establish neither universal superiority nor production-speed gains. Chronological placement provides no lifetime load guarantee.

Generality is limited by short direct-call traces, unresolved Ethereum receipt validation, absent historical-balance reconstruction, in-memory queries, and an allocation-only TxAllo comparison. Future work will study receipt-aware archive replay, general read/write sets, indexed queries, migration, and protocol integration.

References

[1] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," Ethereum Yellow Paper, 2014. [Online]. Available: https://ethereum.github.io/yellowpaper/paper.pdf

[2] M. S. Chishti and A. Banerjee, "Increasing TPS rate of state-based blockchains by parallel mining," *Internet Technology Letters*, vol. 4, no. 2, p. e220, 2021.

[3] Z. Zhen, X. Wang, X. Yang, J. Shu, J. Hu, H. Lin, and X. Yi, "SemantiChain: A trust retrieval blockchain based on semantic sharding," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 10 339–10 354, 2024.

[4] Y. Zhang, S. Pan, and J. Yu, "TxAllo: Dynamic transaction allocation in sharded blockchain systems," in *2023 IEEE 39th International Conference on Data Engineering*, ser. ICDE '23. IEEE, 2023, pp. 721–733.

[5] X. Wang, B. Li, L. Jia, and Y. Sun, "Orbit: A dynamic account allocation mechanism in sharding blockchain system," in *2024 IEEE 44th International Conference on Distributed Computing Systems*, ser. ICDCS '24. IEEE, 2024, pp. 333–344.

[6] Y. Zhang, S. Pan, and J. Yu, "Mosaic: Client-driven account allocation framework in sharded blockchains," in *2025 IEEE 45th International Conference on Distributed Computing Systems*, ser. ICDCS '25. IEEE, 2025, pp. 604–614.

[7] Z. Hong, S. Guo, E. Zhou, J. Zhang, W. Chen, J. Liang, J. Zhang, and A. Y. Zomaya, "Prophet: Conflict-free sharding blockchain via byzantine-tolerant deterministic ordering," in *IEEE INFOCOM 2023–IEEE Conference on Computer Communications*. IEEE, 2023, pp. 1–10.

[8] Y. Liu, L. Jia, X. Yang, Z. Li, and Y. Sun, "Concordia: Enabling low-conflict distributed transaction scheduling in sharding blockchain via cooperative perception," in *Proceedings of the ACM Web Conference 2026*, ser. WWW '26. ACM, 2026, pp. 5493–5502.

[9] T. Dickerson, P. Gazzillo, M. Herlihy, and E. Koskinen, "Adding concurrency to smart contracts," in *Proceedings of the 36th ACM Symposium on Principles of Distributed Computing*, ser. PODC '17. ACM, 2017, pp. 303–312.

[10] R. Gelashvili, A. Spiegelman, Z. Xiang, G. Danezis, Z. Li, D. Malkhi, Y. Xia, and R. Zhou, "Block-STM: Scaling blockchain execution by turning ordering curse to a performance blessing," in *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, ser. PPoPP '23. ACM, 2023, pp. 232–244.

[11] M. Bartoletti, L. Galletta, and M. Murgia, "A theory of transaction parallelism in blockchains," *Logical Methods in Computer Science*, vol. 17, no. 4, pp. 10:1–10:30, 2021.

[12] M. J. Amiri, D. Agrawal, and A. El Abbadi, "ParBlockchain: Leveraging transaction parallelism in permissioned blockchain systems," in *2019 IEEE 39th International Conference on Distributed Computing Systems*, ser. ICDCS '19. IEEE, 2019, pp. 1337–1347.

[13] Z. Chen, X. Qi, X. Du, Z. Zhang, and C. Jin, "PEEP: A parallel execution engine for permissioned blockchain systems," in *Database Systems for Advanced Applications*, ser. DASFAA '21. Springer, 2021, pp. 341–357.

[14] T. Xu, Y. Zhong, Y. Zhang, R. Xiong, J. Zhang, G. Xue, and S. Liu, "Vegeta: Enabling parallel smart contract execution in leaderless blockchains," in *22nd USENIX Symposium on Networked Systems Design and Implementation*, ser. NSDI '25. USENIX Association, 2025, pp. 795–811. [Online]. Available: https://www.usenix.org/conference/nsdi25/presentation/xu-tianjing

[15] S. Blackshear, A. Chursin, G. Danezis, A. Kichidis, L. Kokoris-Kogias, X. Li, M. Logan, A. Menon, T. Nowacki, A. Sonnino, B. Williams, and L. Zhang, "Sui Lutris: A blockchain combining broadcast and consensus," in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '24. ACM, 2024, pp. 2606–2620.

[16] M. J. Amiri, D. Agrawal, and A. El Abbadi, "SharPer: Sharding permissioned blockchains over network clusters," in *Proceedings of the 2021 International Conference on Management of Data*, ser. SIGMOD '21. ACM, 2021, pp. 76–88.

[17] J. Wang and H. Wang, "Monoxide: Scale out blockchains with asynchronous consensus zones," in *16th USENIX Symposium on Networked Systems Design and Implementation*, ser. NSDI '19. USENIX Association, 2019, pp. 95–112. [Online]. Available: https://www.usenix.org/conference/nsdi19/presentation/wang-jiaping

[18] L. N. Nguyen, T. D. T. Nguyen, T. N. Dinh, and M. T. Thai, "OptChain: Optimal transactions placement for scalable blockchain sharding," in *2019 IEEE 39th International Conference on Distributed Computing Systems*, ser. ICDCS '19. IEEE, 2019, pp. 525–535.

[19] M. Król, O. Ascigil, S. Rene, A. Sonnino, M. Al-Bassam, and E. Rivière, "Shard scheduler: Object placement and migration in sharded account-based blockchains," in *Proceedings of the 3rd ACM Conference on Advances in Financial Technologies*, ser. AFT '21. ACM, 2021, pp. 43–56.

[20] H. Huang, X. Peng, J. Zhan, S. Zhang, Y. Lin, Z. Zheng, and S. Guo, "BrokerChain: A cross-shard blockchain protocol for account/balance-based state sharding," in *IEEE INFOCOM 2022–IEEE Conference on Computer Communications*. IEEE, 2022, pp. 1968–1977.

[21] C. Curino, Y. Zhang, E. P. C. Jones, and S. Madden, "Schism: A workload-driven approach to database replication and partitioning," *Proceedings of the VLDB Endowment*, vol. 3, no. 1–2, pp. 48–57, 2010.

[22] G. Karypis and V. Kumar, "A fast and high quality multilevel scheme for partitioning irregular graphs," *SIAM Journal on Scientific Computing*, vol. 20, no. 1, pp. 359–392, 1998.